

BeSecured: a local cybersecurity risk scanner

P12 progress report, week 8 of 10

Mohammedreda El Aroui (project manager), Sidi Hamadi,
Philemon Vittet, Abdelrahmn Dahy

21 June 2026

BeSecured looks at the security settings of a personal computer and turns them into a score out of 100, with a short list of things to fix. It is meant for people who are not security experts. This note says where we are at week 8 of 10.

1 Where we are

The hard part is done. The scanner runs on Windows, macOS and Linux, the score is computed and explained, and the interface works locally from start to finish. Weeks 1 to 7 are finished. Week 8, the current one, is mostly packaging and tests: getting the app to open with a double click, and closing the last gaps in the test suite. The two remaining weeks are kept for the optional plain-language explanations and the final delivery.

Week	Focus	State
1–4	Charter, planning, design, environment	done
5	Cross-platform scanner	done
6	Scoring engine	done
7	Local interface and report export	done
8	Packaging and tests	in progress
9–10	Optional AI explanations, final delivery	to do

One thing changed since the previous report. We had planned a React and Tailwind front end with Chart.js. In the end we dropped all of that. The tool is now a plain Python package with no external dependency, and the interface is hand-written HTML and CSS served locally. For a tool whose whole point is to stay on the machine, fewer moving parts is the right trade.

2 What we built

We started from a PowerShell script that only worked on Windows. It is now a Python package. A single entry point runs the checks that apply to any system, then the ones specific to the current OS, and collects the results into one score.

Each OS has its own module. On Windows we read the firewall, Defender, updates, local users and password policy, shared folders, startup entries, UAC and BitLocker. macOS covers the firewall, FileVault, Gatekeeper, SIP, XProtect, sharing and login items. Linux covers the firewall, updates, LUKS, users, password rules, shares, startup and antivirus. The ports and the basic system information are read the same way everywhere.

A check ends as OK, a warning, or critical; a few are marked info or skipped. Only the first three touch the score. A warning costs 2 points and a critical one costs 5, and the final score is

100 minus the share of points lost:

$$\text{score} = 100 - \text{round}\left(\frac{\text{points lost}}{\text{points possible}} \times 100\right)$$

A skipped check, for example one that needs administrator rights we do not have, is shown as skipped and never counted as a pass. We would rather show a gap than hide it. The report lists, for each finding, what was seen, why it matters, and the steps to fix it.

3 A real scan

Figure 1 is a scan run on one of our macOS machines. It scored 92 out of 100, grade A. No critical issue, two warnings (an XProtect signature a couple of weeks old, and a public shared folder), and the rest fine. The interface opens in its own window, without a browser address bar, which is part of why it no longer feels like a web page.

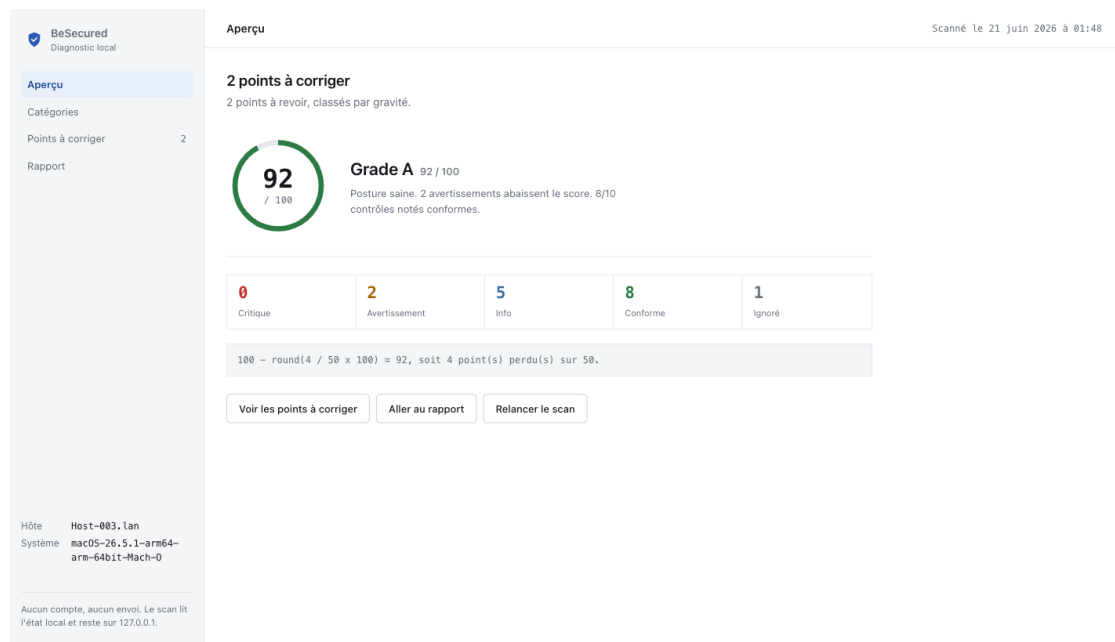


Figure 1: Overview after a scan: the score, the breakdown by status, and the formula that produced it. Everything shown was read on the local machine.

4 Privacy and tests

Nothing leaves the machine. There is no account and no server to talk to: the interface listens on 127.0.0.1 and the scan is local. Reports are written to a local folder, and the program refuses to save one onto a cloud-synced path such as iCloud or OneDrive. A test even checks that the interface files hold no external link.

The project has 59 automated tests. They cover the parsers for each OS, the scoring and its calculation steps, the HTML and JSON export, and the contract between the scanner and the interface. They all pass.

Risk	How we handle it
Some checks need admin rights	Detected at runtime; the check is skipped with a note instead of failing.
False positives could mislead a non-expert	Unavailable checks are skipped, not passed; the result shape is guarded by tests.
Sensitive data must stay local	Local-only design, 127.0.0.1, the cloud-path guard, and a test for external links.
The optional AI part could slip	It stays optional and gets dropped first if week 8 or 9 runs late.

5 Risks and what is left

What is left is short. Week 8: package the app for Windows and macOS so it opens with a double click, run the full test pass, and do a last privacy check. Week 9: the optional plain-language explanations for the top findings, generated locally if we can. Week 10: the demo, the documentation and the handoff.